Module 7 Week B — Applied Lab: Pre-Trained QA Evaluation on Tech/Entertainment News

In this lab you will evaluate a pre-trained extractive Question-Answering (QA) model on a ~1,000-example curated tech-news QA set, compute exact-match (EM) and token-F1, identify three categorical failure modes by inspecting the lowest-F1 examples, and produce a one-page per-task evaluation report. This is the second concrete artifact of Module 7's "fine-tuned model + evaluation report" — Week A produced the fine-tuned classifier; this lab produces the QA half of the integrated comparison.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

What You Will Produce

Committed to your repo:

1. `lab.py` — your implementation of the QA evaluation pipeline.
2. `qa_metrics.json` — aggregate EM, token-F1, n, and the model id.
3. `qa_predictions.csv` — per-question predictions with EM and F1.
4. `qa-evaluation-report.md` — a one-page report (structure given below).

No model file — pre-trained QA models are loaded from Hugging Face Hub at runtime. Nothing is saved or committed.

Setup

1. **Accept the assignment**
2. **Clone your repo:**

```
git clone <your-repo-url>
cd <your-repo-name>
```

3. **Install dependencies:**

```
pip install -r requirements.txt
```

This installs `transformers`, `torch` (CPU build), `pandas`, `numpy`, `pytest`, and `huggingface_hub`. No new packages compared to the drill (the drill installed `rouge-score`, which the lab does not use — that is the integration task's territory).

4. **Create your working branch:**

```
git checkout -b lab-7b-pretrained-qa
```

All implementation work goes in `lab.py`. The starter file contains function signatures and docstrings — implement each as described below.

> *Note on first run:* `lab.py` *constructs a QA pipeline, which downloads the model weights (~250 MB) on first run. Plan ~3 minutes for the first run; subsequent runs use cached weights. The full evaluation on ~1,000 examples completes in 4–6 minutes on CPU after the model is cached.*

What You're Building

`lab.py` is one Python script that runs the full QA evaluation end-to-end. The structure mirrors the Reading: load examples → build pipeline → predict → score → write artifacts.

You will implement the following functions. The first three are normalization and metric functions you also wrote in the drill — feel free to copy from your drill solution:

```
get_data_path()                 # provided helper
get_qa_model_name()             # provided helper
load_examples(data_path)        # provided helper — pd.read_csv + schema check
normalize_answer(...)           # TODO (same as drill1)
exact_match(...)                # TODO (same as drill1)
token_f1(...)                   # TODO (same as drill1)
build_qa_pipeline(...)          # TODO (same as drill1)
predict_one(...)                # TODO (new — wraps the pipeline call)
evaluate_qa(...)                # TODO (new — orchestrates the harness)
main()                          # TODO (orchestration)
```

The starter `lab.py` includes:

- **`get_data_path()`** — returns the `DATA_PATH` env var if set (CI smoke fixture), otherwise `"data/tech_news_qa.csv"`. The autograder uses `DATA_PATH` to switch between the smoke fixture (CI) and the real ~1,000-row evaluation set (your local runs).
- **`get_qa_model_name()`** — returns the hardcoded `"distilbert-base-cased-distilled-squad"` model id. The smoke-test threshold requires the real model, so CI does not substitute a smaller checkpoint here.
- **`load_examples(data_path)`** — `pd.read_csv` over the path, with a small schema check that the four required columns (`qid`, `question`, `context`, `gold_answer`) are present. The full CSV ships a fifth column (`article_id`) which the harness ignores.

Do not modify these helpers. The autograder depends on them.

Task 1: Normalization and Metric Functions

Implement `normalize_answer(s)`, `exact_match(pred, gold)`, and `token_f1(pred, gold)` exactly as you did in the drill (you may copy your drill solution). Refer to the Reading § 4 for the SQuAD-style normalization spec and the F1 edge cases (empty handling).

These are the foundation of the harness. The autograder verifies them against fixture cases before checking pipeline functions.

Task 2: Build the QA Pipeline

Implement `build_qa_pipeline(model_name)`:

```
def build_qa_pipeline(model_name: str):
    """Construct a Hugging Face question-answering pipeline."""
    # TODO: return pipeline("question-answering", model=model_name)
```

Same as the drill. The model name is read from `get_qa_model_name()`.

Task 3: Predict One Answer

Implement `predict_one(qa, question, context)`:

```
def predict_one(qa, question: str, context: str) -> str:
    """
    Run the QA pipeline on one (question, context) pair.

    Returns the answer string only (not the full pipeline output dict).
    """
    # TODO: call qa(question=..., context=...)
    # TODO: extract and return the "answer" field
```

Why a thin wrapper? The pipeline returns a dict. The evaluation harness only needs the answer string for EM/F1, isolating that extraction in `predict_one` keeps `evaluate_qa` clean.

Task 4: Evaluate Over the Dataset

Implement `evaluate_qa(qa, examples)`:

- `examples` is a Pandas DataFrame with columns `qid`, `question`, `context`, `gold_answer`.
- For each row, call `predict_one`, compute EM and F1.
- Return a dictionary:

```
{
    "em": float,            # mean EM across examples
    "f1": float,            # mean token-F1 across examples
    "n": int,               # number of examples
    "predictions": [        # one entry per example
        {
            "qid": str,
            "question": str,
            "context_excerpt": str,    # first 80 chars of context
            "gold_answer": str,
            "predicted_answer": str,
            "em": int,
            "f1": float,
        },
        ...
    ],
}
```

```
def evaluate_qa(qa, examples) -> dict:
    """
    Evaluate the QA pipeline over a DataFrame of examples.

    Returns a dict with aggregate metrics and per-question predictions.
    """
    # TODO: iterate over examples; call predict_one, compute em + f1
    # TODO: collect into the predictions list
    # TODO: return {em, f1, n, predictions}
```

Why include `context_excerpt`? When a learner reviews predictions in `qa_predictions.csv`, having an 80-char excerpt of the context makes it possible to spot-check predictions without the full context column blowing out the spreadsheet width.

Task 5: Review `main()` (already wired up)

`main()` is already provided in `lab.py` (lines 126–152). It loads the data via `load_examples(get_data_path())`, constructs the pipeline via `build_qa_pipeline(get_qa_model_name())`, calls `evaluate_qa`, then writes:

- `qa_predictions.csv` (columns: `qid, question, context_excerpt, gold_answer, predicted_answer, em, f1`)
- `qa_metrics.json` (`{em, f1, n, model}`)

You do **not** need to rewrite `main()`. Open it, read it, and confirm you understand the flow. Your remaining TODOs in `evaluate_qa` (Task 4) drive the artifact writes — once those are correct, `python lab.py` produces both files.

Do **not** write `qa-evaluation-report.md` from `main()` — you will write that markdown file by hand based on the metrics and the CSV.

The Evaluation Report

After running `python lab.py`, you will have `qa_predictions.csv` and `qa_metrics.json`. Now write `qa-evaluation-report.md` (a one-page markdown report) with the following sections:

1. **Dataset description (1–2 sentences).** What data, what slice, where it came from.
2. **Model.** Name and Hugging Face Hub URL.
3. **Aggregate metrics.** EM and token-F1, both reported to two decimal places. Note the gap between them (or the lack of gap) — what does that pattern suggest?
4. **Failure-mode taxonomy.** Identify **three** categorical failure modes by sorting `qa_predictions.csv` by `f1` ascending and inspecting the lowest-F1 examples. Each failure mode should be:
   - Named clearly (e.g., "off-by-article boundary," "span includes trailing punctuation," "model picks distractor entity").
   - Distinct from the other two (not three flavors of the same thing).
   - Illustrated with **one concrete (qid, question, gold_answer, predicted_answer) example**.
5. **Domain judgment.** Pick one realistic deployment domain for extractive QA (financial filings, medical literature, legal contracts, customer documentation, etc.) and write 2–3 sentences on whether you would ship this model for that domain — naming a specific consideration (latency, faithfulness, calibration, no-answer support, content sensitivity).

The report should fit comfortably on one page. Don't pad. Don't restate what's in the JSON.

Submitting

When all functions are implemented, the local tests pass, and your evaluation report is written:

1. **Stage and commit:**

```
git add lab.py qa_metrics.json qa_predictions.csv qa-evaluation-report.md
git commit -m "Complete Lab 7b"
```

2. **Push:**

```
git push -u origin lab-7b-pretrained-qa
```

3. **Open a Pull Request** from your branch into `main`. The autograder runs automatically.

Your PR description must include:

1. Dataset description (1–2 sentences).
2. Final aggregate EM and token-F1 (two decimal places each).
3. Names of the three failure modes you identified.
4. One sentence on whether you would ship this model for production QA in your selected domain.
5. Paste your PR URL into TalentLMS → Module 7 → Lab 7b to submit this assignment.

Peer Review (Wed–Thu)

The Day 5 weekly minimum includes one peer review. Week B's peer review is on **this lab's evaluation report**.

- **Wednesday morning:** an assignment script pairs you with one teammate's Lab 7B PR.
- You review their `qa-evaluation-report.md` against the **5-item checklist** below.
- Submit your review as a comment on their PR by **Thursday end of day**.

Peer Review Checklist (5 items)

For each item, write a sentence or two. Don't just check yes/no — the reviewer's value is in their reasoning, not in the binary judgment.

1. **Aggregate metrics.** Are EM and token-F1 stated with two-decimal precision and the model named?
2. **Three distinct failure modes.** Are the three failure modes clearly different categories — not three flavors of the same thing?
3. **Concrete example per mode.** Does each failure mode include an actual `(question, gold, predicted)` excerpt that genuinely illustrates the category, not a random low-F1 pick?
4. **Domain judgment.** Does the "would you ship this?" paragraph name a specific, technical consideration (latency, faithfulness, calibration, no-answer support, content sensitivity) — not just yes / no?
5. **Reproducibility.** Does the PR include `qa_predictions.csv` (~1,000 rows) and `qa_metrics.json`, with the report's headline numbers matching the JSON?

You do **not** re-run the autograder, hand-check F1, or re-construct the pipeline. The autograder and TA cover those.

After completing your review, paste a link to your review comment into TalentLMS → Module 7 → Peer Review 7B.

Challenge Extensions

The following extensions are optional; they extend the lab but do not replace any required deliverable. They are graded as part of the lab's rubric portion when the base assignment is complete.

Challenge Tier 1 — Model comparison

Evaluate a second QA model on the same ~1,000-example set. Recommended: `deepset/roberta-base-squad2` (which supports a "no answer" prediction — though the curated tech-news QA set always has an answer, so the no-answer support is unused on this slice). Produce a side-by-side metrics table and write 2–3 sentences interpreting where each model wins. Add a section to your evaluation report: **Model Comparison**.

Challenge Tier 2 — Long-context handling

Construct a 5-example test set where each context is artificially extended to >512 tokens (concatenate the original tech-news context with several other articles' text — your gold answer should still be in the original passage). Document how the pipeline handles overflow (truncation? automatic windowing?), measure how F1 changes vs. the short-context baseline, and write a 1-page memo (`long-context-memo.md`) on the production implications. Useful for understanding what happens when production passages exceed the model's max length.

Challenge Tier 3 — Manual QA inference loop

Reproduce the QA pipeline using `AutoModelForQuestionAnswering` directly: tokenize the question + context, run the model, extract start/end logits, compute the best span subject to `max_answer_length`, decode the span, and post-process. Compare predictions to the pipeline's predictions on a 30-example subset. A senior engineer should need 4+ hours; requires understanding token-to-character mapping, special-token handling, and span-constraint logic. Produce a `manual_qa.py` and a `manual-vs-pipeline-memo.md` documenting any divergences.

What's Next

- **Tomorrow (Tuesday) — Honors Track only:** Stretch — Adversarial QA Probe (extends this lab on branch `stretch/lab-adversarial-qa`)
- **Wednesday (Day 4):** Integration Task — pre-trained summarization on the tech news corpus + the integrated evaluation report (the M7 deliverable).
- **Thursday (Day 5):** Peer review (this lab's report) due; integration submission due; reflection; readiness checks.